

Engineering Documentation: AI/ML Phase 1, Week 2

Document Status: Approved & Completed

Author: Jainish Mehta

Focus: Relational Database Architecture, SQL Query Optimization, ETL Pipelines, and Data Analytics.

Executive Summary

The objective of Week 2 was to transition the data storage architecture from static, flat-file arrays (CSVs) into highly normalized Relational Database Management Systems (RDBMS). Before deploying Machine Learning models, an engineer must guarantee data integrity, perform server-side aggregations, and automate data ingestion. This week focused on architecting Third Normal Form (3NF) databases, writing secure Data Manipulation scripts, mastering Relational Set Theory (JOINS), and building two automated, production-grade ETL and Analytics pipelines.

Technology Stack

- **IDE:** Visual Studio Code (VS Code) with SQLite Viewer
- **Database Engine:** SQLite3 (Serverless embedded SQL engine)
- **Version Control:** Git & GitHub
- **Language:** Python 3.x
- **Libraries:** `sqlite3`, `requests`, `pandas`, `matplotlib`, `Pillow` (PIL), `subprocess`

Daily Execution Log

Day 1 & 2: Database Architecture & Setup

- **Objective:** Deprecate flat files and establish a strictly typed relational storage layer.
- **What was done:**
 - Authored standard SQL Data Definition Language (DDL) schemas utilizing `CREATE TABLE`.

- Implemented strict schema constraints including `PRIMARY KEY AUTOINCREMENT` , `NOT NULL` , and `UNIQUE` .
- Developed a Python driver script using the `sqlite3` library to execute raw `.sql` files, programmatically generating the `company.db` file without manual terminal intervention.
- **Why we did it (The Rationale):**
 - CSV files lack internal validation and are prone to duplicate or malformed data. By establishing an RDBMS, we offload data validation (type checking, uniqueness) to the C-kernel database engine, ensuring 100% data integrity before extraction to the Python application.

Day 3 & 4: Secure CRUD, Idempotency & Aggregations

- **Objective:** Safely manipulate database states and optimize analytical queries to prevent RAM exhaustion.
- **What was done:**
 - Replaced insecure Python string interpolation with **Parameterized Queries** (? placeholders) for all `UPDATE` and `DELETE` operations.
 - Engineered Pipeline Idempotency utilizing `INSERT OR IGNORE` and `INSERT OR REPLACE` conflict resolution.
 - Shifted mathematical operations from Python to SQL utilizing `SUM()` , `AVG()` , `GROUP BY` , and `HAVING` .
- **Why we did it (The Rationale):**
 - Parameterized queries physically separate SQL execution plans from user inputs, neutralizing the threat of **SQL Injection (SQLi)**. Furthermore, transferring computational workloads (Aggregations) to the database engine minimizes network latency and prevents Application Out-of-Memory (OOM) crashes on large datasets.

Day 5 & 6: 3NF Normalization & Set Theory (JOINS)

- **Objective:** Eliminate data duplication through Relational Algebra and table fragmentation.
- **What was done:**
 - Normalized databases to **Third Normal Form (3NF)** by abstracting transactional data into associative mapping tables.

- Enforced Referential Integrity by activating `FOREIGN KEY` constraints with `ON DELETE CASCADE`.
 - Executed complex relational traversals utilizing `INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN`, `CROSS JOIN`, and `FULL OUTER JOIN`.
 - Engineered Python data-marshalling algorithms to safely cast SQL `NULL` values to safe defaults, preventing `NoneType` rendering crashes.
- **Why we did it (The Rationale):**
 - Storing categorical text inside transactional tables creates update anomalies. Abstracting data into linked tables prevents redundancy. SQL Joins act as the bridge, seamlessly stitching the optimized hardware architecture back into a human-readable format dynamically in RAM.

Day 7: Automated ETL & Fraud Detection (Final Projects)

- **Objective:** Synthesize all API, Python, and SQL knowledge into two end-to-end Enterprise pipelines.
- **What was done:**
 - **Project 1 (Automated Crypto Warehouse):** Engineered a Time-Series Database mapping a `coins` dimension table to a `price_history` fact table. Converted web logos into binary `BLOB` formats for local SQLite storage, and mapped the data directly into a dynamic `matplotlib` dashboard displaying the raw BLOB images. Automated the pipeline using a `subprocess` daemon.
 - **Project 2 (E-Commerce Fraud Engine):** Architected an E-commerce schema and simulated thousands of random transactions. Wrote advanced query analytics to automatically flag Fraudulent Users (using `HAVING` high-spend bounds) and identify Customer Churn (using `LEFT JOIN` Anti-Joins).
- **Why we did it (The Rationale):**
 - Demonstrates the ability to handle both Time-Series (FinTech) and Relational Transactional (E-commerce) data models. Building an automated daemon script proves readiness for modern continuous-operation MLOps and Data Engineering roles.

Key Deliverables Achieved

1. **The SQL Masterclass & Ultimate Cheat Sheet:** Authored 6 highly detailed, L4-level Systems Architecture technical specifications.

2. **Binary Data Operations:** Successfully parsed, stored, and retrieved raw Binary Large Objects (BLOBs) directly via SQL.
3. **Two Production-Grade Data Pipelines:** Built automated ETL architectures handling both API ingestion (Upserts) and algorithmic Anomaly Detection (Fraud tracking).

Next Steps You are 100% correct, and I sincerely apologize.

I looked at the completion of the Database Project and prematurely jumped to the end of the Phase. Looking directly at the Master Gantt Chart we created, **Phase 1 spans 10 entire weeks.**

We just finished Week 2. We still have Math, Pandas, EDA, Supervised ML, Unsupervised ML, and the monumental Phase 1 Final Project remaining. You are keeping us perfectly on track.

Fix your WEEK_2_OVERVIEW.md

Please change the very last section of your Week 2 Overview document to this:

Next Steps (Week 3 Preview)

With the data extraction and architecture pipelines fully mastered, Week 2 concludes. We are currently 20% through **Phase 1: The Data & ML Foundation**. The roadmap now shifts to **Week 3: Applied Linear Algebra**. Before utilizing advanced Machine Learning libraries, an engineer must understand the core mathematics driving them. Week 3 will focus on Vectors, Matrices, Dot Products, and Matrix Multiplication executed entirely through Python code.